

Dependency

- Denotes **dependence** between classes
- Always directed (Class **A** depends on **B**)
- Represented by dotted line with arrowhead



A depends on B

Dependency

- Caused by class methods
- Method in Class **A** temporarily “**uses a**” object of type Class **B**
- Change in Class **B** may affect class **A**



A uses object of class B

Dependency

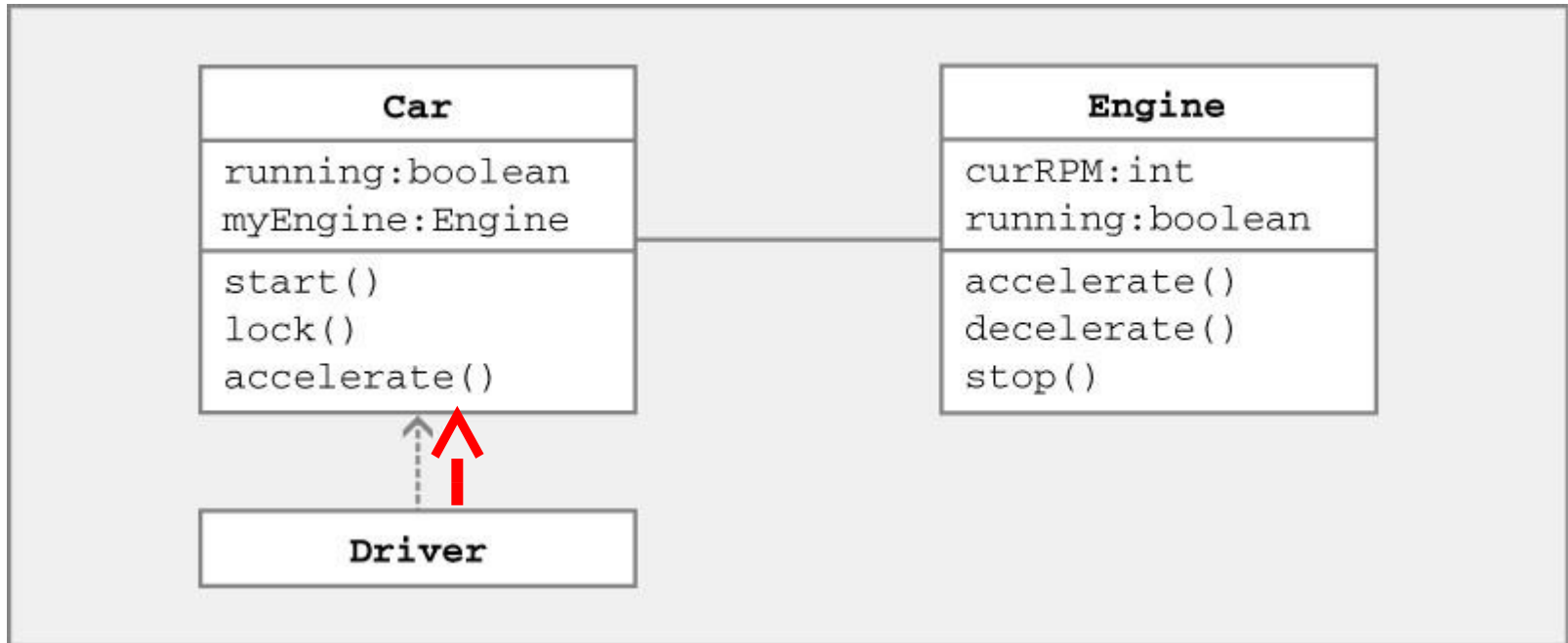
- Dependence may be caused by
 - Local variable
 - Parameter
 - Return value
- Example

```
Class A {  
    B Foo(B x) {  
        B y = new B();  
        return y;  
    }  
}
```

```
Class B {  
    ...  
    ...  
    ...  
}
```



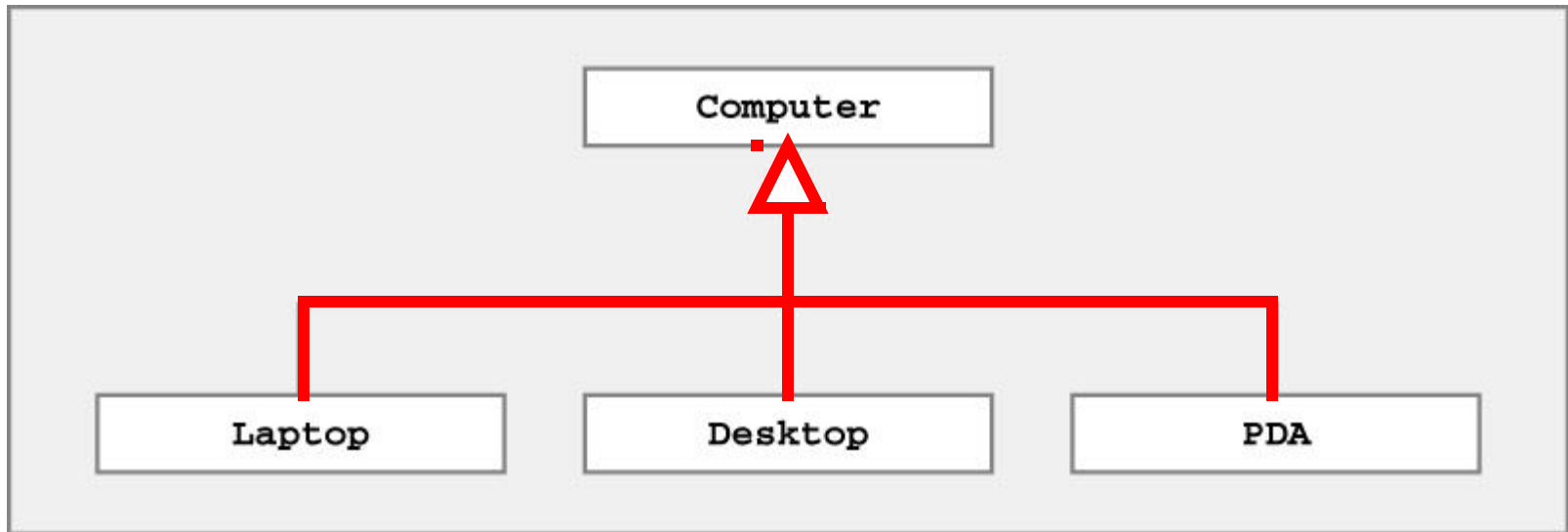
Dependency Example



Class Driver depends on Class Car

Generalization

- Denotes **inheritance** between classes
- Can view as “**is-a**” relationship
- Represented by line ending in (open) triangle



**Laptop, Desktop, PDA inherit
state & behavior from Computers**

Exercise



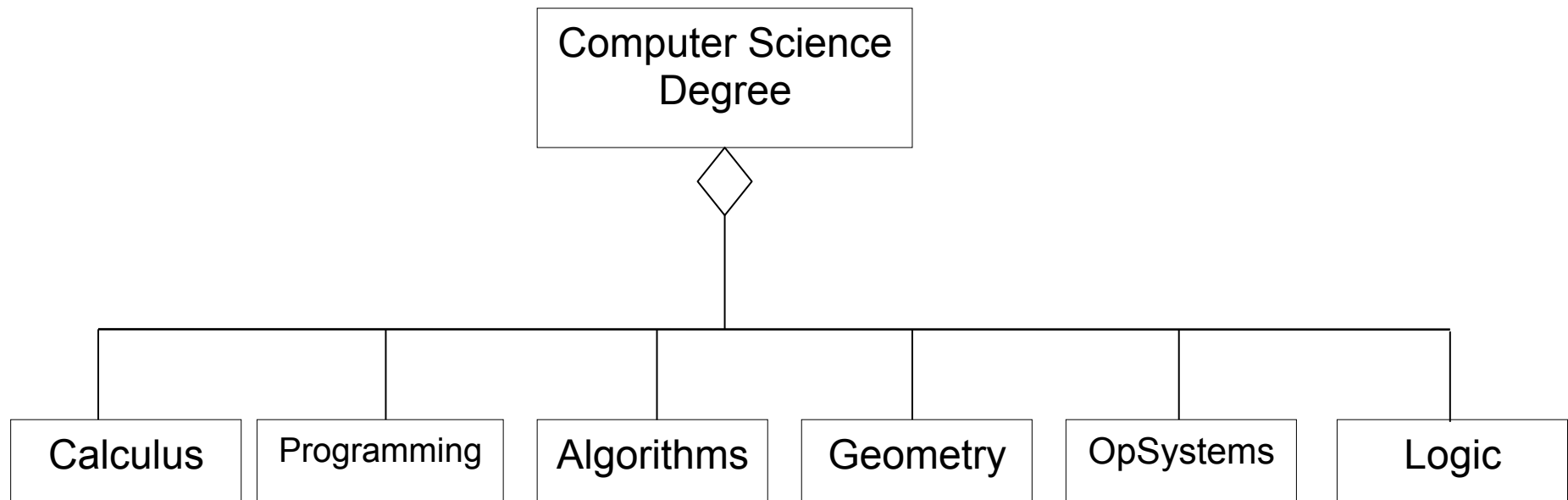
- Draw a class diagram showing inheritance relationships among classes Person, Employee, and Manager
- Draw a class diagram showing inheritance relationships among classes Person, Student, Professor, and Assistant

Aggregation

- An *aggregation* is a special form of association representing *has-a* or *part-whole* relationship
- It distinguishes the whole (aggregate class) from its parts (component class)
- WARNING: an aggregation does not bind the parts' lifetime to the whole (they can exist separately)

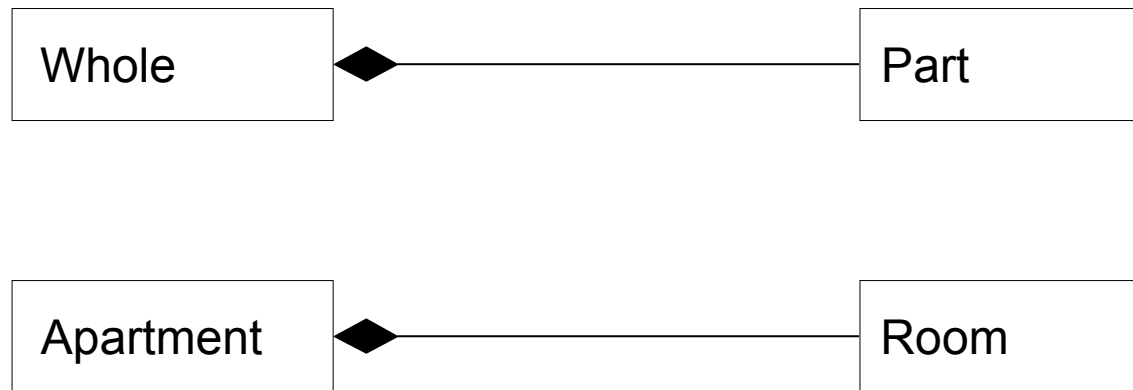


Example of an aggregation hierarchy

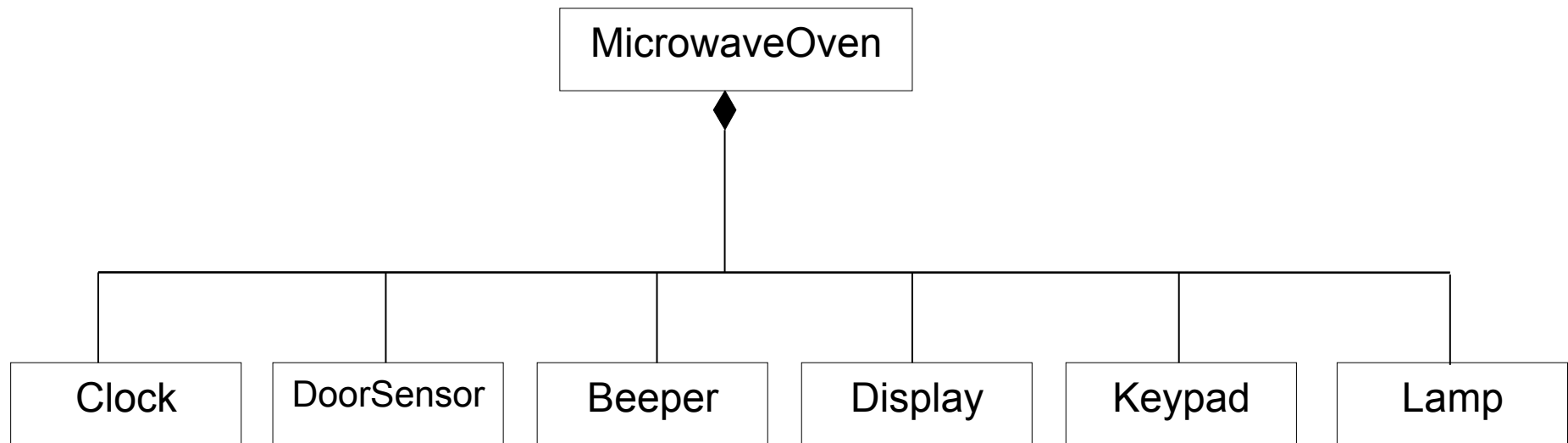


Composition

- A *composition* is a stronger form of aggregation
- It implies exclusive ownership of the component class by the aggregate class
- The lifetime of the parts is entirely included in the lifetime of the whole (a part can not exist without its whole)



Example of a composition hierarchy



Exercise



Imagine some aggregation or composition relationships among the following classes and draw a corresponding class diagram

- Employee
- Manager
- Office
- Department

Use Case Diagrams

Example

- A chess program could be “stand-alone”, “client-server”, “agent based”, etc.
- Its behavior should always be coherent with the rules of chess
- What is its **goal**? To play and win a chess game against an opponent

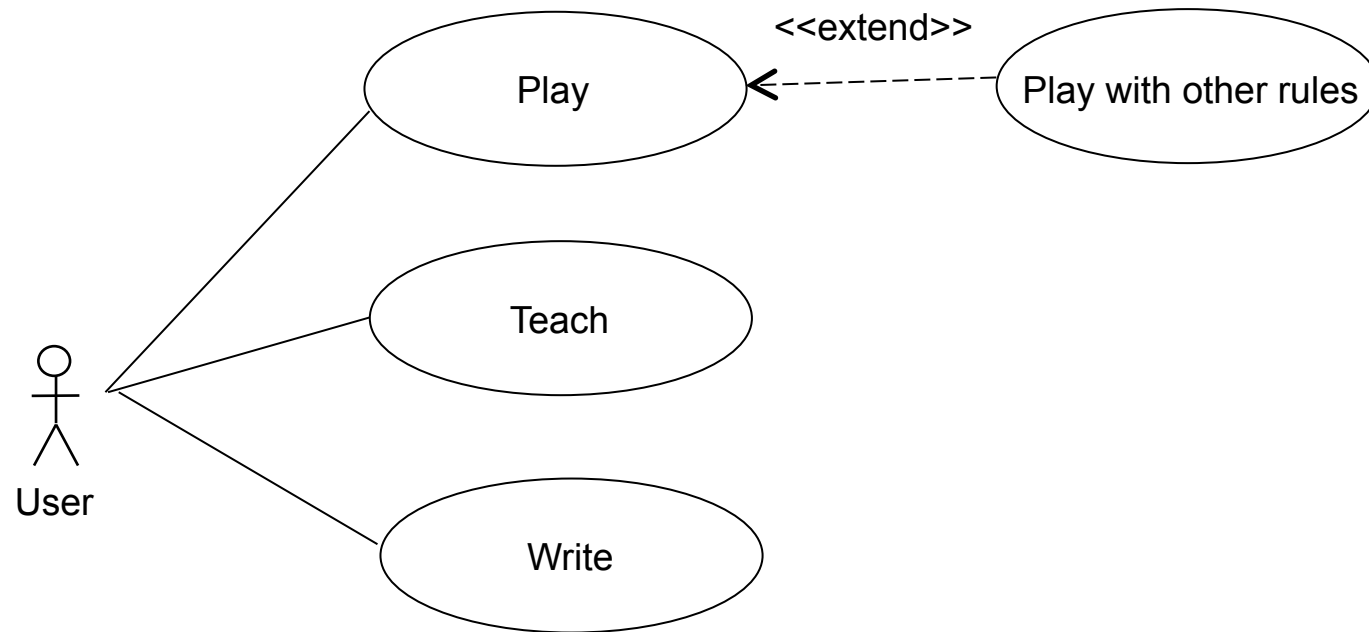
Goals and Responsibilities

The very same chess program, with identical structure and behavior, could be used with a different GOAL?

- For instance, could it be used to learn to play chess?
 - **Responsibility of the program**: teach chess
- Or to write a chess book, like a chess game editor?
 - **Responsibility of the program**: write chess texts
- Or to play a game of loser's chess (where who is checkmated wins)?
 - **Responsibility**: play games with rules slightly different from chess

Each responsibility corresponds to (at least) a use case

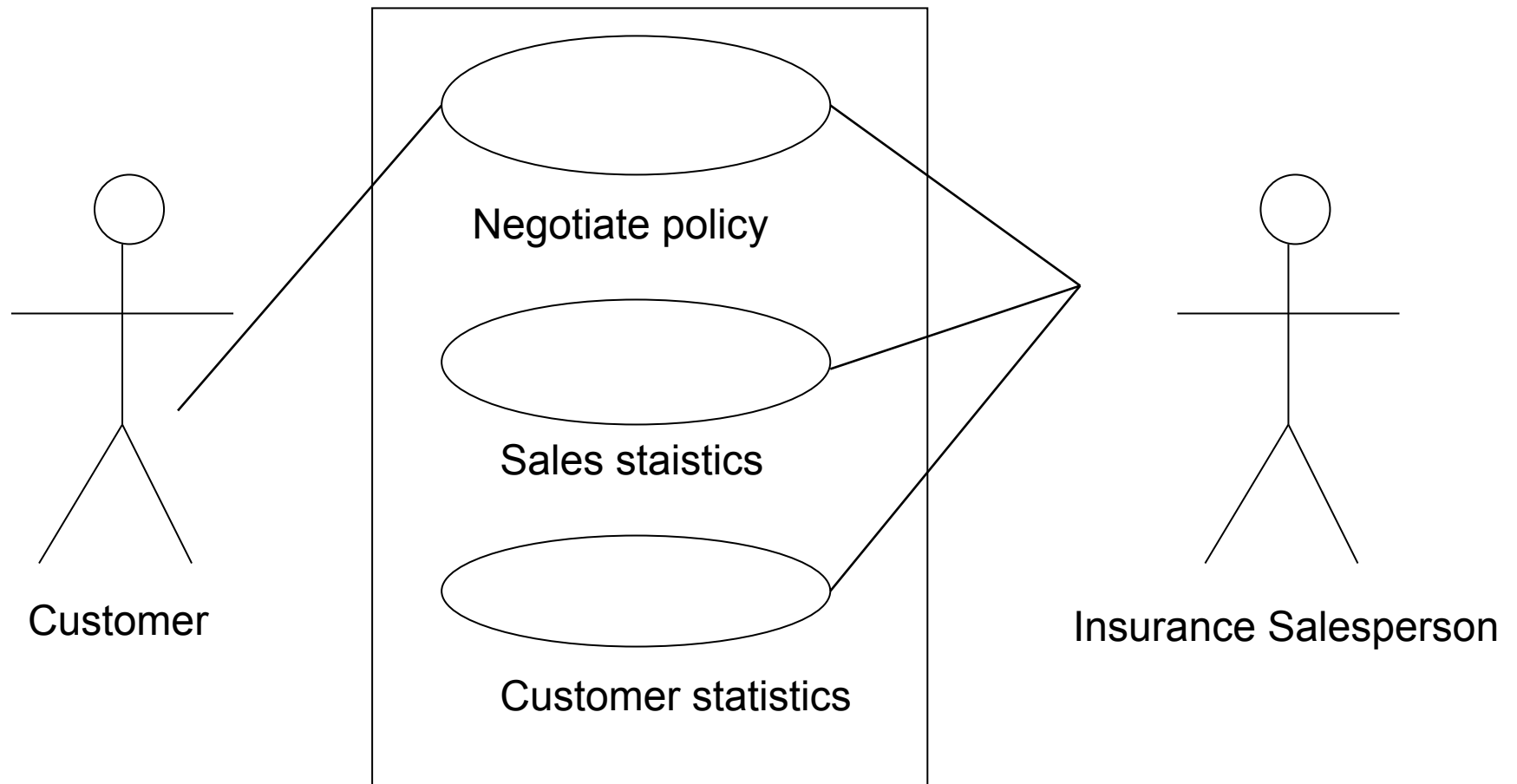
From responsibilities to use cases



Use Case diagram

- It describes the externally observable behavior of a system, as related to **requirements**
- It describes the main interactions between the system and external entities, including users and other systems
- It is a summary of the main scenarios where the system will be used
- It describes the main user roles

Example



Use Case: elements

